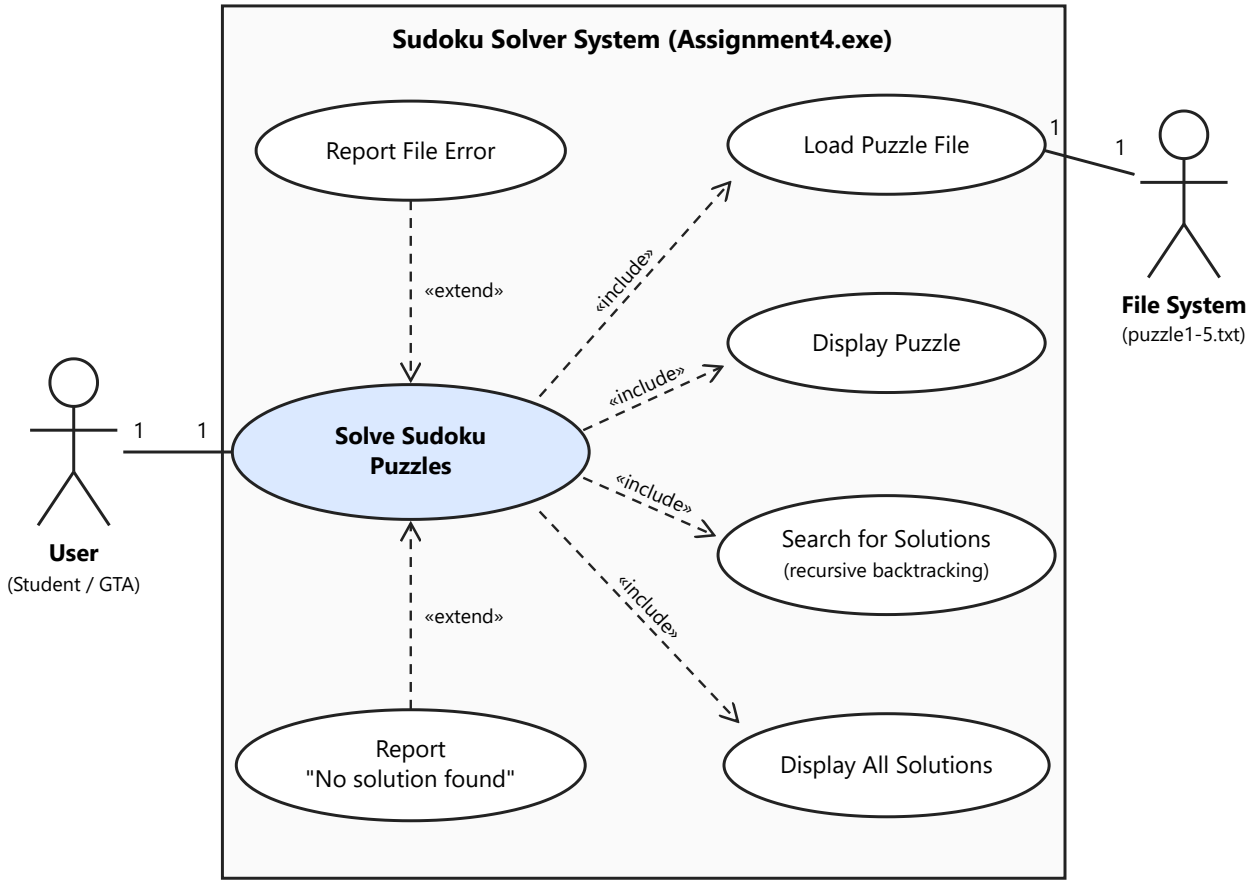
EECS 348 Assignment 4 – UML Use Case Diagram

System: Recursive Object-Oriented Sudoku Solver (Assignment4.exe) | Author: Justin Whitaker | Date: 10/15/2026

Prepared with help from a GenAI (Claude Opus, via the Google Antigravity IDE) and reviewed by the author.

Use Case Diagram



Notation: solid lines are associations (multiplicity 1 at each end: one user runs one execution of the program, and the program reads from one file system). Dashed «include» arrows point from the base use case to behavior that always happens. Dashed «extend» arrows point from optional/exceptional behavior back to the base use case it extends.

Step 1 – Identify / Discover the Use Case

System	Sudoku Solver System (the Assignment4.exe program)
Primary actor	**User** – the person (student or GTA) who runs the program on the EECS cycle server.
Secondary actor	**File System** – external storage that supplies puzzle1.txt – puzzle5.txt.
Base use case	**UC-1 Solve Sudoku Puzzles** – goal: "*As a User, I want to solve the five Sudoku puzzle files so that I can see every valid solution (or learn that none exists).*"

Included use cases	Load Puzzle File, Display Puzzle, Search for Solutions, Display All Solutions (always performed for every file).
Extending use cases	Report File Error (only when a file is missing or malformed); Report "No solution found" (only when the search finds zero solutions).

Step 2 – Brief Description

This use case allows a User to solve the Sudoku puzzles stored in puzzle1.txt through puzzle5.txt. For each file, the system reads the puzzle from the File System, prints the file name and the original puzzle, and then uses a recursive depth-first search with backtracking to find every solution. Each solution is printed along with the total number of solutions found; if a puzzle has no solution, the system prints "No solution found". If a file cannot be read or is badly formatted, the system reports the error and continues with the next file.

Step 3 – Step-by-Step Flow (Outline) of Events

1. User starts the program (`./Assignment4.exe`).
2. System selects the next puzzle file (puzzle1.txt first).
3. System prints the puzzle file name.
4. System requests the file from the File System.
5. File System returns the file contents.
6. System reads and validates the 81 cells.
7. System prints the original puzzle.
8. System checks that the given clues do not break any Sudoku rule.
9. System recursively searches for solutions (find empty cell → list candidates → try each → backtrack).
10. System prints each solution as it is found.
11. System prints the total number of solutions found.
12. System repeats steps 2–11 for the remaining files.
13. System ends the program.

Step 4 – Detailed Description

Use case name	UC-1 Solve Sudoku Puzzles
Primary actor	User (student or GTA running the program)
Secondary actor	File System (provides puzzle1.txt – puzzle5.txt)
Stakeholders & interests	**Student:** wants correct output for all five puzzles to earn full credit. **GTA / Instructor:** wants clear, labeled output that is easy to check against known solutions.
Trigger	The User runs `./Assignment4.exe` from a terminal on the EECS cycle server.
Preconditions	1. Assignment4.exe has been compiled on the cycle server. 2. puzzle1.txt – puzzle5.txt are in the same directory the program is run from. 3. Each file is expected to hold 81 space-separated entries, each either a digit 1–9 or an underscore (_) for a blank.
Postconditions (success)	For every file, the file name, the original puzzle, and either every solution plus "Solutions found: N" or "No solution found" have been printed. The puzzle files are unchanged.
Postconditions (failure)	For a file that cannot be read, an error message is printed and the remaining files are still processed.
Main success scenario	1. The User starts the program. 2. The system creates a PuzzleRunner object containing the five file names. 3. For the next file name, the system prints a separator line and "Puzzle file: <name>". 4. **«include» Load Puzzle File:** the system asks the File System to open the file, reads exactly 81 entries, converts '_' to an empty cell and each digit to its value, and confirms there is no extra data. 5. **«include» Display Puzzle:** the system prints the puzzle exactly as it appears in the file. 6. The system confirms that no given clue repeats a digit within its row, column, or 3×3 block. 7. **«include» Search for Solutions:** starting at the first cell, the system finds the next empty cell and builds a list of candidate digits not already used in that cell's row, column, or block. It places the first candidate and recursively continues with the next empty cell. When a cell has no candidates, the system backtracks by removing the last guess and trying the next candidate. 8. **«include» Display All Solutions:** each time all 81 cells are filled, the system prints "Solution k:" and the completed grid, then keeps searching so that every solution is found. 9. When the search has tried every possibility, the system prints "Solutions found: N". 10. Steps 3–9 are repeated for puzzle2.txt through puzzle5.txt. 11. The program exits normally (return code 0).

Alternative flows / extensions	**4a. «extend» Report File Error – file is missing:** the system prints "Error: Could not open <name>" and continues at step 10 with the next file. **4b. «extend» Report File Error – bad entry:** an entry is not '_' or a single digit 1–9; the system prints "Error: Invalid entry '<entry>' in <name>" and continues at step 10. **4c. «extend» Report File Error – wrong size:** the file has fewer or more than 81 entries; the system prints the matching error and continues at step 10. **6a. «extend» Report "No solution found" – conflicting clues:** a given clue repeats a digit; the search is skipped, the system prints "No solution found", and continues at step 10. **9a. «extend» Report "No solution found" – search fails:** every path ends in a cell with no candidates (zero solutions, e.g. puzzle4.txt); the system prints "No solution found" instead of the solution count. **8a. Multiple solutions:** the puzzle has more than one solution (e.g. puzzle5.txt has 2); each one is printed in the order found and the count reflects all of them.
Special requirements	Must be written in C++ as a single file (Assignment4.cpp) using objects and a recursive algorithm; must compile and run on the EECS cycle servers; all five puzzles should finish in well under one second.
Frequency of use	Once per run; each run processes all five puzzle files.
Assumptions	The User has terminal access to the cycle server and permission to read the puzzle files. No interactive input is needed after the program starts.